Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ème Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 06

Enseignant : Benyamina Ahmed

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

III- Algorithmes de tri

IV- Les arbres

V- Les graphes

1. Définition
2. Représentation des graphes
3. Parcours des graphes
4. Etude de complexité

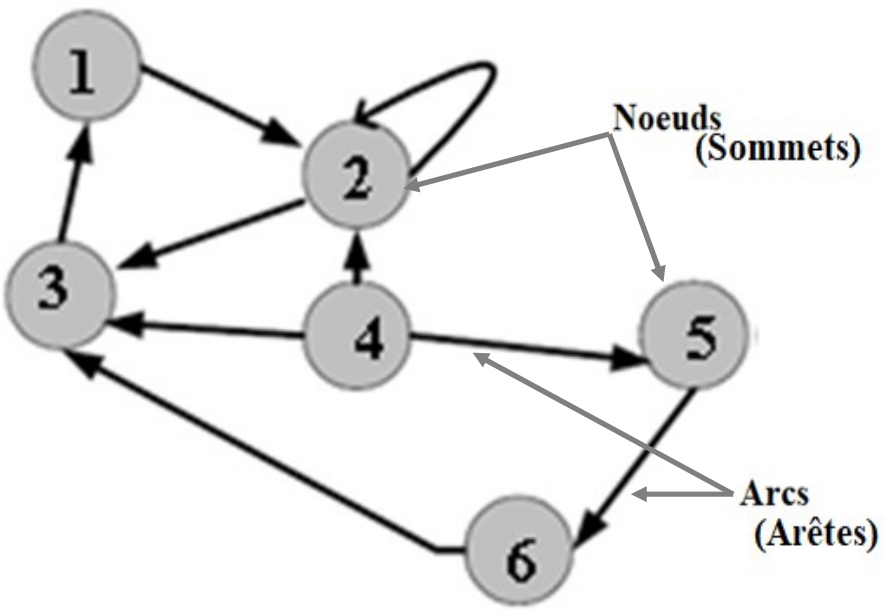
Les graphes

1.Définitions

Les graphes: Un **graphe** est une structure de données qui permet de représenter des **objets** et des **relations** entre eux. Un **graphe** est composé de **sommets** (ou **nœuds**) et **d'arêtes** (ou **arcs**) qui relient les sommets, c'est-à-dire, Le graphe G = (V, E), où V représente l'ensemble des sommets et E représente l'ensemble des arêtes.

Exemples: On peut utiliser un graphe pour modéliser:

1. **Un réseau social**: pour modéliser les relations sociales sur les plateformes numériques, **(+ détail)**
2. **Un réseau routier**: en représentant les connexions entre intersections
3. **Un circuit électrique**: en analysant la circulation du courant entre composants;

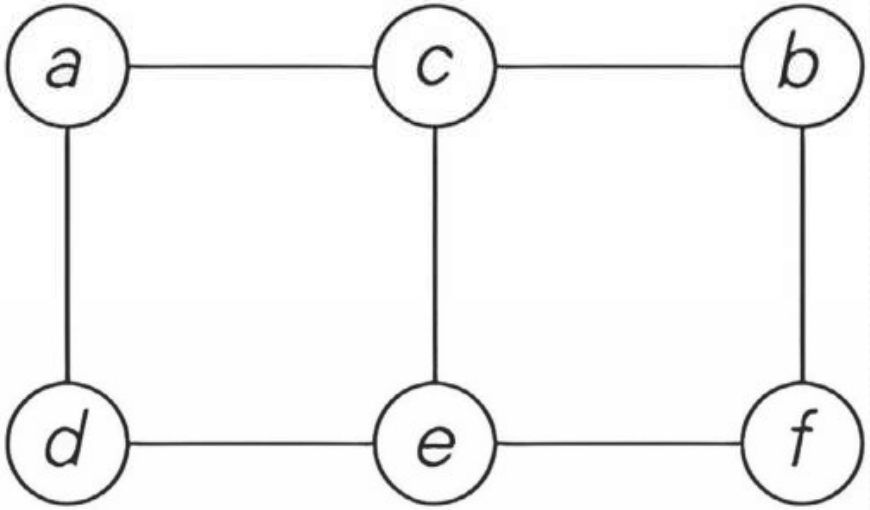


1.Définitions : types de graphes

-Il existe deux types de graphes : les graphes non orientés et les graphes orientés.

1. Graphe non orienté:

Les arêtes n'ont pas de sens, c'est-à-dire qu'on peut parcourir une arête dans les deux sens.



Le graphique représenté à la figure a six sommets et sept arêtes non orientées :

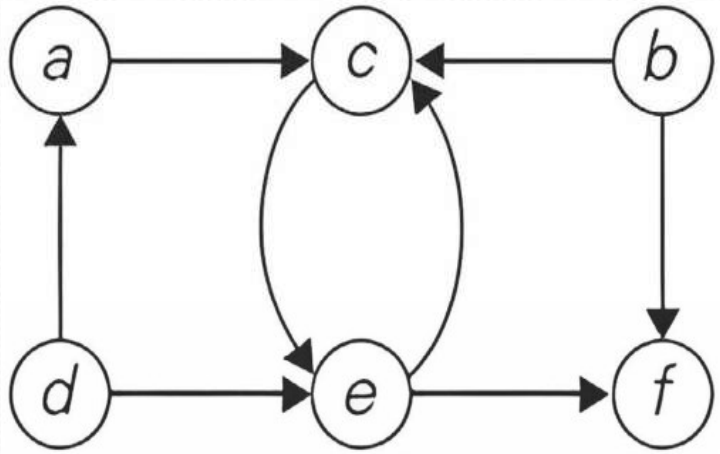
V = {a, b, c, d, e, f },

E = {(a, c), (a, d), (b, c), (b, f), (c, e), (d, e), (e, f)}

$$Cardinalité \ : |V| = N = 6 \quad ; \quad |E| = M = 7$$

-Il existe deux types de graphes : les graphes non orientés et les graphes orientés.

2. Graphe orienté:



Les arêtes ont un sens, c'est-à-dire qu'on ne peut parcourir une arête que dans une seule direction.
On représente les arêtes orientées par des flèches.

Le digraphe représenté à la figure comporte six sommets et huit arêtes dirigées :

V = {a, b, c, d, e, f },

E = {(a, c), (b, c), (b, f), (c, e), (d, a), (d, e), (e, c), (e, f)}

Cardinalité : $|V| = N = 6$; $|E| = M = 8$

Les graphes

1.Définitions

1. **Ordre du graphe**: C'est le nombre de sommets du graphe.

Exemple: L'ordre du graphe 1 est 4,

Celui du graphe 2 est 3.

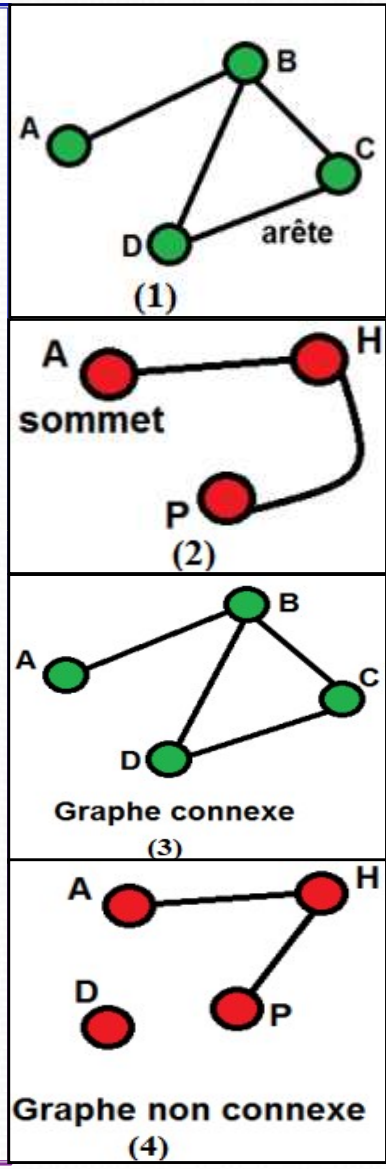
2. **Degré** : le degré d'un sommet est défini comme le nombre d'arêtes incidentes à ce sommet.

Exemple : Le degré du sommet A est 1, le degré du sommet B est 3, C et D sont de degré 2 (graphe 2 ou 3). Le degré du sommet D du graphe 4 est 0

Objectif: Le calcul du degré est pour identifier les sommets isolés (degré zéro), les sommets terminaux (degré un), les séquences de degrés, et d'autres caractéristiques structurales des graphes.

3. **Connexe:** Un graphe est dit **connexe** s'il n'y a pas de point isolé.

Exemple: Le graphe 4 n'est pas connexe car le sommet D est isolé



Les graphes

1.Définitions

1. Graphe symétrique: Si toutes ces arêtes orientés peuvent se parcourir dans les 2 sens.

Exemple: - Le graphe 1 est symétrique,

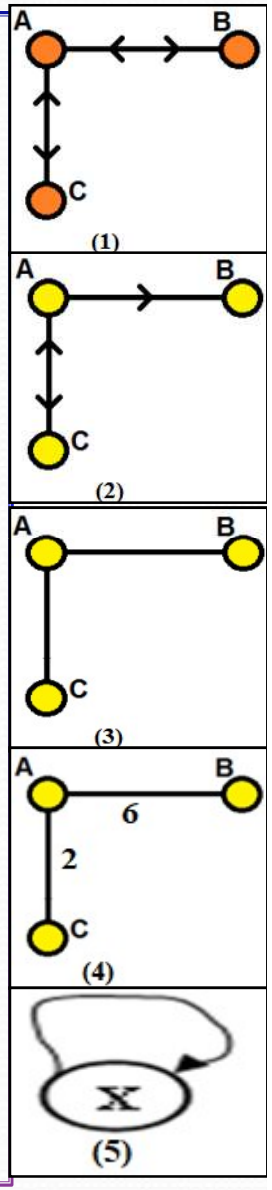
- Celui du graphe 2 ne l'est pas car l'arc AB existe mais pas l'arc BA
- un graphe 3 non-orienté est symétrique car les arêtes peuvent se parcourir dans les 2 sens.

2. **Sommets adjacents:** deux sommets sont dits adjacents s'il existe une arête ou un arc les reliant : peu importe qu'il soit orienté ou non.

Exemple: B et A sont adjacents, mais B et C ne le sont pas

3. Graphe valué: Un graphe valué est un ensemble de connexions entre des sommets, où chaque connexion a un nombre associé, représentant une mesure telle que la distance, le coût, ou une autre valeur pertinente.

4. Boucle: Est un arc reliant un nœud à lui même. Graphe 5.



Les graphes

1.Définitions

1. **Graphe complet**: Un graphe est **complet** si tous les sommets sont adjacents, c'est-à-dire que tout les sommets sont reliés deux à deux entre eux.

Exemple: - Le graphe 1 est complet,

- Celui du graphe 2 ne l'est pas car l'arc AB car il n'y a aucune arête entre B et C :

2. **Chemin**: suite d'arcs (orienté) ou de connexions (non orienté) consécutifs qui relient deux nœuds.

Exemple: Chemin de A et D : **({A, B}, {B,D}) ou ({A, D})**

Un chemin est dit **simple** s'il ne rencontre pas deux fois le même sommet.

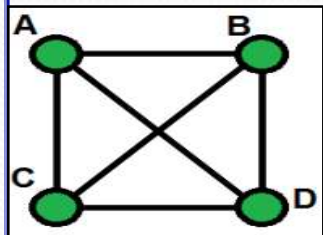
3. Circuit: est un chemin fermé. Le nœud initial c'est le nœud final.

Exemple: ({A, B}, {B,D}, {D,A})

4. Longueur d'un chemin: est le nombre d'arcs constituant un chemin.

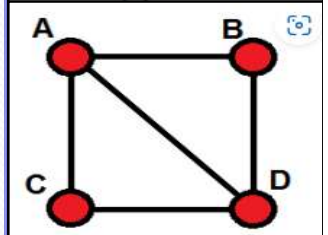
Exemple : Le chemin {A, B}, {B,D} est de longueur 2

Le chemin {A, D} est de longueur 1



Graphe complet

(1)



Graphe non complet

(2)

Les graphes
1.Définitions

<u>Les graphes:</u> Un **graphe** est une structure de données qui permet de représenter des **objets** et des **relations** entre eux.

Exemples: on peut utiliser un graphe pour modéliser:

1. **Un réseau social**: pour modéliser les relations sociales sur les plateformes numériques,
2. **Un réseau routier**: en représentant les connexions entre intersections
3. **Un circuit électrique**: en analysant la circulation du courant entre composants;
4. **La biologie moléculaire**: en décrivant les interactions entre molécules dans une cellule;
5. **Guidance Digitale Personnalisée** : en modélisant les préférences des utilisateurs et les relations entre produits pour une recommandation efficace

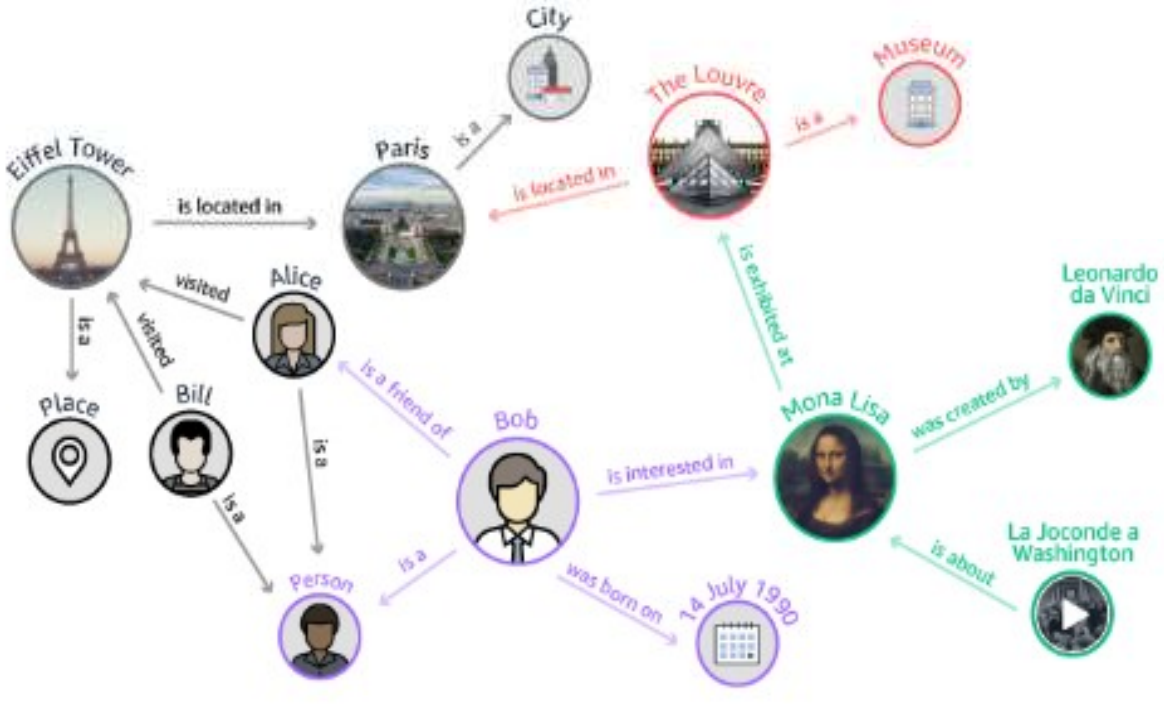
etc.

Les graphes:

1. **Un réseau social**: pour modéliser les relations sociales sur les plateformes numériques.

De tels graphes permettent ainsi aux sites de réseaux sociaux de vous proposer de nouveaux contacts en fonction de vos envies et de vos connaissances.

D'après le diagramme précédent, quelle visite pourrait-on conseiller à Bob ?



Les graphes

1.Définitions

Les graphes: Après avoir vu les notions des graphes revenons à les voir dans l'exemple de **réseau social**

Concepts	Définitions	Dans les réseaux sociaux
Sommet	Point du graphe	Personne inscrite
Arête	Lien dans le graphe	Relation entre deux personnes
Degrés	Nombre de sommets connectés à un sommet donné	Nombre d' "amis"
Distance entre deux sommets	Nombre minimal d'arêtes pour relier deux sommets	Nombre de contact séparant deux personnes
Excentricité d'un sommet	Distance maximale entre un sommet et l'ensemble des autres sommets du graphe	Nombre de contacts séparant une personne du reste du réseau
Rayon d'un graphe	Valeur minimale de l'excentricité	Identifie les personnes influentes dans le réseau
Centre(s) d'un graphe	Sommet(s) ayant l'excentricité minimal	
Diamètre d'un graphe	Distance maximale entre deux sommets d'un graphe	Degré de séparation dans le réseau

Conclusion: Ces concepts clés dans les réseaux sociaux fournissent une perspective structurée pour comprendre les relations, les influences et les dynamiques au sein de ces espaces interconnectés.

2.Représentation des graphes

➢**Représentation d'un graphe :** Il existe plusieurs types de représentations des graphes.

Parmi ces représentations on a les deux types suivants:

1. Matrice d'adjacence,
2. Matrice d'incidence.
3. Représentation chaînée

2.Représentation des graphes

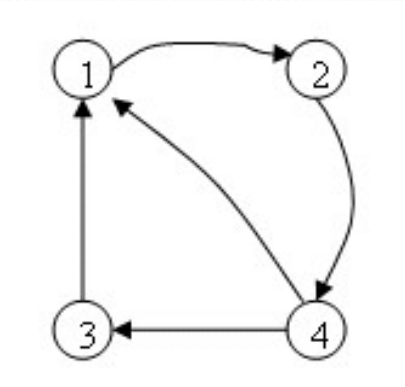
➢**Représentation d'un graphe :**

1. Matrice d'adjacence:

La matrice d'adjacence, est une matrice, qui fait correspondre les sommets origines des arcs (placés en ligne) aux sommets destination (placés en colonnes).

L'existence d'un arc (x_i,x_j) se traduit par la présence d'un 1 à l'intersection de la ligne x_i et de la colonne x_j; l'absence d'arc par la présence d'un 0.

Exemple : Le graphe suivant est représenté par la matrice d'adjacence :



Sommets	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	1	0	0	0
4	1	0	1	0

L'inconvénient majeur est la consommation de mémoire quelque soit le graphe (pour **V nœuds** on a **V^2 places mémoires**)

2.Représentation des graphes

➢**Représentation d'un graphe :**

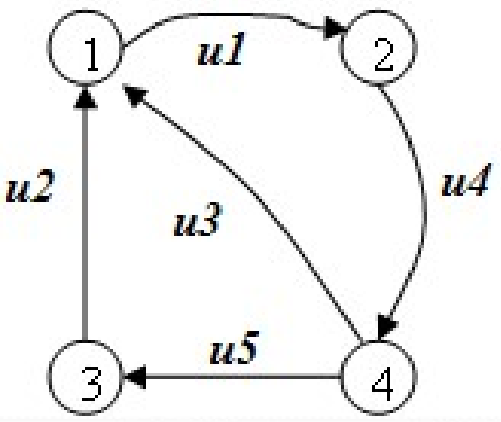
2. Matrice d'incidence:

La matrice d'incidence est une matrice qui fait correspondre les sommets (placés en ligne) aux arêtes (placées en colonnes). Chaque ligne représente un sommet et chaque colonne représente une arête. Les éléments de la matrice indiquent la participation des sommets aux arêtes.

Une entrée de cette matrice:

(sommet,arc) = (i,uj) vaut 0 ou 1 : $= \begin{cases} 1 & \text{si l'arc commence en i} \\ -1 & \text{si l'arc se termine en i} \\ 0 & \text{sinon} \end{cases}$

Exemple : Le graphe suivant est représenté par la matrice d'incidence :



Sommets \ Arcs	*u1*	*u2*	*u3*	*u4*	*u5*
1	**1**	**-1**	**-1**	**0**	**0**
2	**-1**	**0**	**0**	**1**	**0**
3	**0**	**1**	**0**	**0**	**-1**
4	**0**	**0**	**1**	**-1**	**-1**

La consommation de mémoire pour V nœuds et M arcs est: **V.M** places

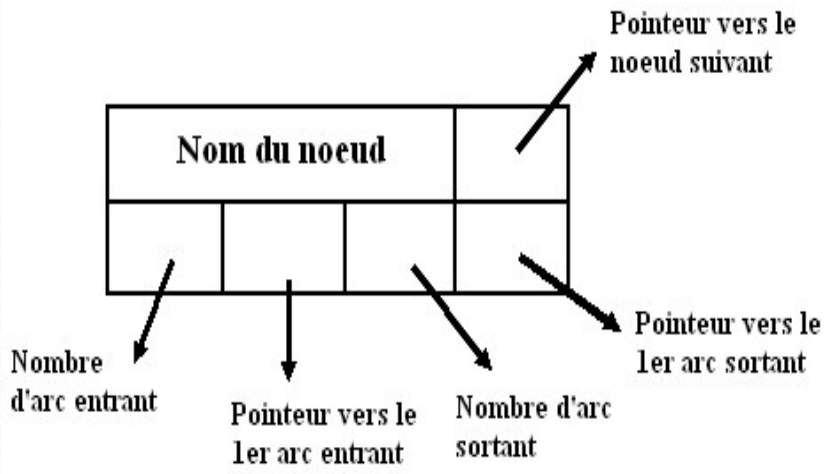
2.Représentation des graphes

➢**Représentation d'un graphe :**

3. Représentation chaînée : Les nœuds et les arcs seront codés sous forme d'enregistrement, avec gestion dynamique de la mémoire.

3.1. Représentation d'un nœud :

Les champs "Nombre d'arcs sortant" et "Nombre d'arcs entrant" sont utilisés lors de la recherche de tous les arcs associés à un nœud.



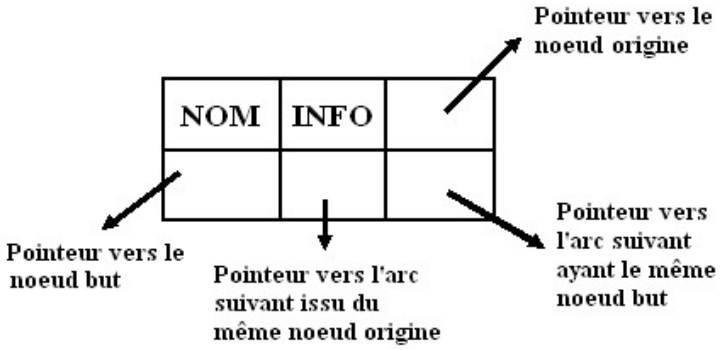
Exemple: Un réseau de transport aérien dans un graphe, les nœuds sont les aéroports desservis.

2.Représentation des graphes

➢**Représentation d'un graphe :**

3. Représentation chaînée : Les nœuds et les arcs seront codés sous forme d'enregistrement, avec gestion dynamique de la mémoire.

3.2. Représentation d'un arc:



Le champ "**Info**" est utilisé dans le cas **valué**.

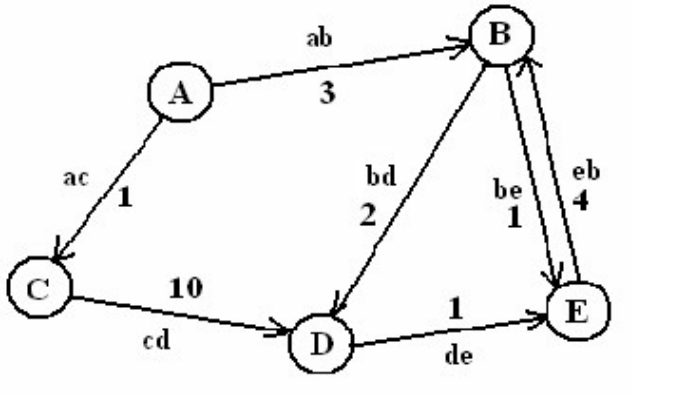
Exemple: Un réseau de transport aérien:
Dans un graphe, les arcs sont les vols entre ceux-ci.
On peut coder en champs "**Info**", les distances entre aéroports

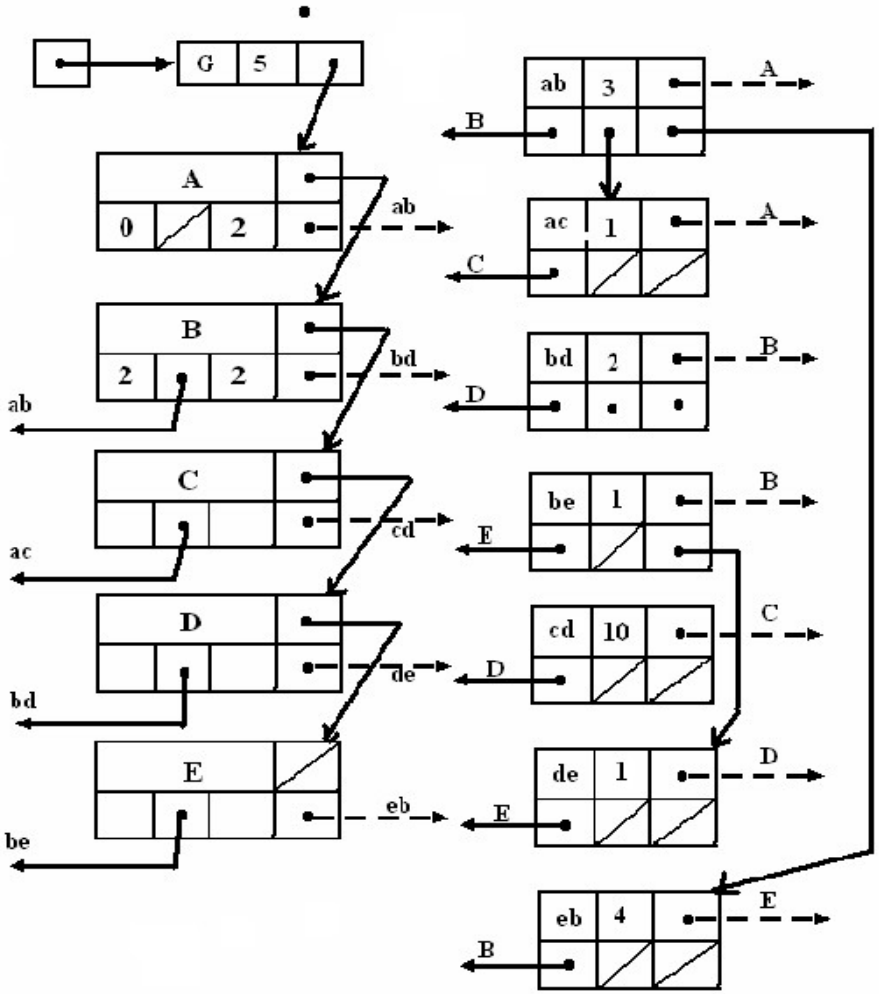
2.Représentation des graphes

➢**Représentation d'un graphe :**

3. Représentation chaînée :

Exemple:





Ce graphe est orienté contenant des arcs valués.

Les nœuds sont chaînés les uns des autres.

Par suite on a une liste chaînée de nœuds.

La notion de premier arc sortant et de premier arc entrant est arbitraire

3.Parcours des graphes

➢**Parcours des graphes :** Le parcours d'un graphe consistant à visiter chaque sommet une et une seule fois. Si le graphe n'est pas connexe, l'algorithme ne pourra parcourir que la composante connexe du sommet de départ.

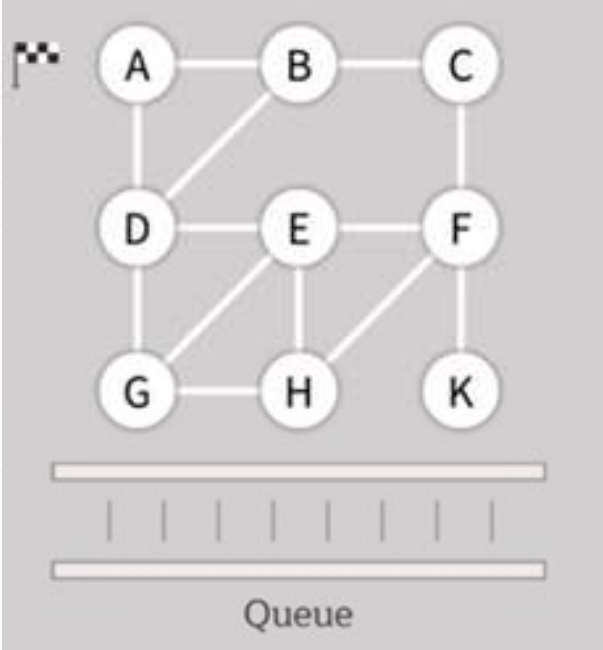
On se limite seulement à voir le parcours dans un graphe représenté par une matrice d'adjacence.
Les types de parcours dans un graphe:

1- Parcours en largeur (Breadth-first Search): BFS
2-Parcours en profondeur (Depth-first Search): DFS

3.Parcours des graphes

➢**Parcours des graphes :**

1- ***Parcours en largeur (Breadth-first Search : BFS)***



Le (BFS) est un algorithme de parcours de graphe. L'algorithme BFS utilise une file d'attente (FIFO) pour empiler les nœuds visités.
Il commence au sommet sélectionné et visite d'abord tous ses voisins, puis tous leurs voisins, et ainsi de suite, jusqu'à ce que tous les sommets du graphe aient été visités.

Remarques: à appliquer durant le parcours.

1. Si le nœud actuel n'a aucun nœud adjacent non visité, l'algorithme supprime le nœud suivant de la file d'attente et en fait le nœud actuel.
2. Si la file d'attente est vide, le parcours est terminé. Tous les nœuds du graphe sont visités.

3.Parcours des graphes

➢Parcours des graphes :

1- *Parcours en largeur (Breadth-first Search : BFS)*

1. Commencez par n'importe quel nœud, marquez-le comme visité, et ajoutez-le à la file d'attente.
2. Tant que la file d'attente n'est pas vide :

a) Retirez le nœud en tête de la file d'attente (défilement ou dépilement).

b) Explorez tous les nœuds adjacents non visités du nœud actuel.

c) Marquez chaque nœud adjacent comme visité et ajoutez-le à la file d'attente.

3. Répétez les étapes 2a à 2c jusqu'à ce que la file d'attente soit vide.

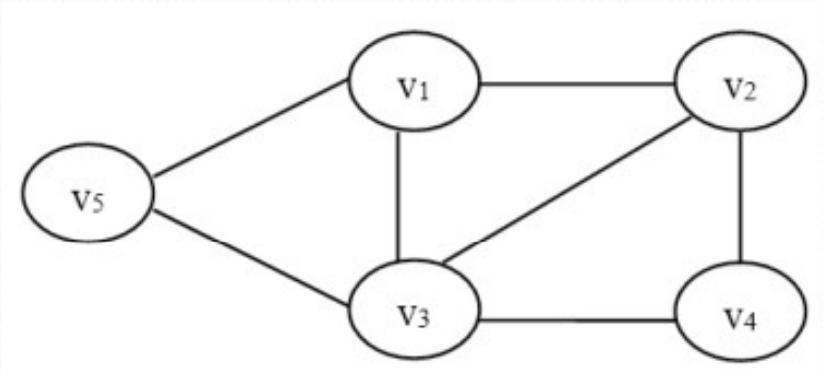
3.Parcours des graphes

➢Parcours des graphes :

1- ***Parcours en largeur (Breadth-first Search : BFS)***

Exemple: Soit le graphe suivant

Ainsi sa matrice d'adjacence



Sommets	v1	v2	v3	v4	v5
v1	0	1	1	0	1
v2	1	0	1	1	0
v3	1	1	0	1	1
v4	0	1	1	0	0
v5	1	0	1	0	0

Etapes	Sommets adjacents dans la file	Sommets visités
1	v1	v1
2	v2, v3, v5 (dépi	
3	v2, v3 (v5 n'a pas de sommets non visité) dépilement de v3)	v1, v2, v3, v5
4	v2, v4 (dépilement de v4)	v1, v2, v3, v5, v4
5	v2 (dépilement de v2)	v1, v2, v3, v5, v4
6	Pile vide	

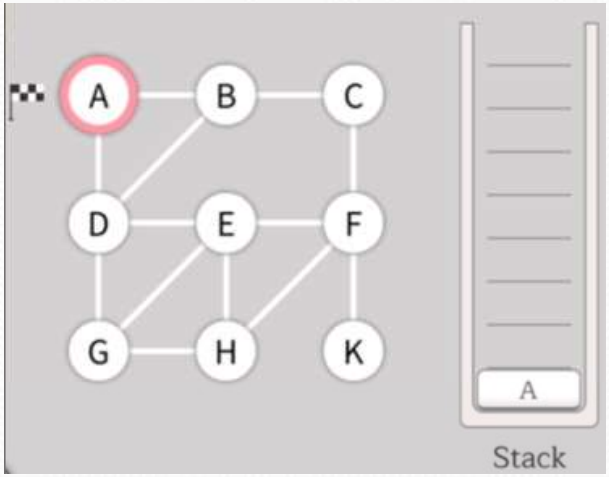
la séquence de parcours est : **v1, v2, v3, v5, v4**

3.Parcours des graphes

➢**Parcours des graphes :**

2- ***Parcours en profondeur (Depth-first Search)***

Depth First Search (DFS) est un algorithme de parcours de graphe qui part d'un sommet sélectionné et visite tous les sommets du graphe en profondeur.



L'algorithme de parcours en profondeur (DFS) utilise une pile (stack en anglais) pour maintenir l'ordre des nœuds à explorer.

La pile suit le principe Last In, First Out (LIFO), ce qui signifie que le dernier nœud ajouté à la pile est le premier à en être retiré.

L'algorithme visite un sommet puis visite tous les voisins du sommet de manière récursive, jusqu'à ce qu'il atteigne la fin d'un chemin ou une impasse.

Si l'algorithme atteint une impasse, il revient au dernier sommet visité et continue le parcours à partir de là.

3.Parcours des graphes

➢**Parcours des graphes :**

2- ***Parcours en profondeur (Depth-first Search)***

1. Commencer par visiter un sommet
   a) Marquer le sommet comme visité.
   b) Empiler le sommet sur une pile.

[redacted]

2. **Si le sommet actuel a un sommet adjacent non visité :**
   a) Empiler le sommet sur la pile.
   b) Marquer le sommet comme visité.

   Sinon :
   a) Dépiler le sommet de la pile.
   b) Rechercher un sommet adjacent non visité à partir du sommet en haut de la pile.
   c) Si un tel sommet est trouvé, le marquer comme visité et l'empiler.

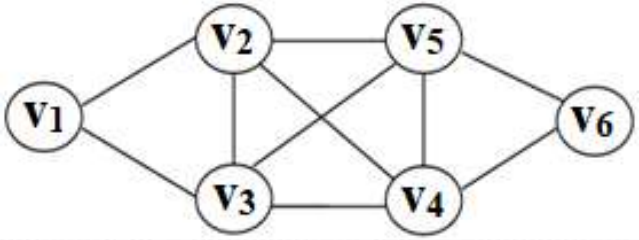
3. Répéter l'étape 2 jusqu'à ce que la pile soit vide.
4. La pile est vide, ce qui signifie que tous les sommets ont été visités.
5. La recherche du sommet adjacent commence depuis le début de la liste de tous les sommets du graphe.

3.Parcours des graphes

➢Parcours des graphes :

1- ***Parcours en largeur (Depth-first Search : BFS)***

Exemple: Soit le graphe suivant
Ainsi sa matrice d'adjacence



Somm	v1	v2	v3	v4	v5	v6
v1	0	1	1	0	0	0
v2	1	0	1	1	1	0
v3	1	1	0	1	1	0
v4	0	1	1	0	1	1
v5	0	1	1	1	0	1
v6	0	0	0	1	1	0

Sommets visités	Les sommets adjacents	Les prochains sommets non visités
$\boxed{V_1}$	V_2, V_3	V_2
$V_1, \boxed{V_2}$		
$V_1, V_2, \boxed{V_3}$	[illegible]	[illegible]
$V_1, V_2, V_3, \boxed{V_4}$	V_2, V_3, V_5, V_6	V_5
$V_1, V_2, V_3, V_4, \boxed{V_5}$	V_2, V_3, V_4, V_6	V_6
$V_1, V_2, V_3, V_4, V_5, \boxed{V_6}$	V_4, V_5	NULL

la séquence de parcours est : **v1, v2, v3, v4, v5, v6**

3.Parcours des graphes : complexités

➢ **Parcours des graphes :** ***Complexité Dans le pire des ca***

Le graphe possède:

V : nbre de sommets
M : nbre d' arêtes

Parcours	Complexité en temps
Parcours en profondeur : Depth-first Search : DFS	**O(V + M)**
Parcours en largeur : Breadth-first Search : BFS	**O(V + M)**

chaque sommet du graphe doit être visité une fois
chaque arête du graphe doit être examinée une fois